



Capstone Project 1 – AI-Enhanced Web Application Task

[Visit our website](#)

Introduction

Congratulations on reaching your first capstone project in the Web Development course. This project is an opportunity to consolidate the front-end development, asynchronous JavaScript, API integration, storage management, and AI interaction design skills you have developed throughout this level of the course.

In earlier lessons, you learned how to:

- Manipulate the Document Object Model (DOM)
- Handle events with JavaScript
- Work with asynchronous operations using `fetch`, `async`, and `await`
- Store and retrieve data using Web Storage APIs
- Integrate AI APIs into web applications
- Design user experiences for AI-powered interactions

This project brings those concepts together into a single practical application.

You will build an AI-enhanced meal ordering application that interacts with TheMealDB API while incorporating AI-inspired interaction patterns such as intelligent recommendations, loading feedback, retry handling, and conversational user feedback. The goal of this project is not simply to retrieve and display data. Instead, you will design an application that feels dynamic, interactive, and responsive to the user.

This project should be treated as a portfolio-quality submission. Employers and technical mentors often evaluate projects not only on whether they function correctly, but also on code quality, usability, structure, maintainability, and user experience design. Take the time to organise your application carefully, write readable code, and build an interface that demonstrates attention to detail.

Developer portfolio considerations

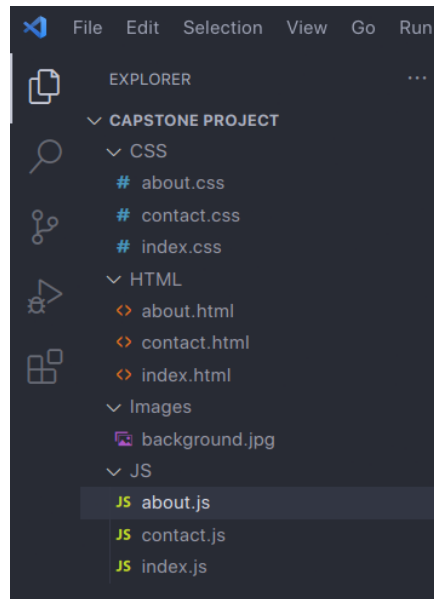
Projects like this are valuable because they demonstrate your ability to combine multiple technologies into a cohesive application. A strong developer portfolio project shows that you can:

- Structure a front-end application logically
- Work with external APIs
- Handle asynchronous workflows
- Manage application state
- Build responsive interfaces
- Design meaningful user interactions
- Handle edge cases and failures gracefully

Modern web applications increasingly integrate AI-powered experiences. Even simple AI-inspired functionality such as recommendation systems, conversational feedback, or intelligent suggestions can significantly improve the perceived quality of an application. This project introduces those ideas at a foundational level while still reinforcing core web development principles.

Before you begin

A major focus of this project is producing clean, maintainable, professional front-end code. Before submitting your work, ensure that your HTML, CSS, and JavaScript are correctly structured, readable, and consistently formatted. Organise your project using a clear directory structure. Your entire application should exist inside a single project folder with meaningful file and folder names. A typical structure may look like this:



You should also make effective use of browser developer tools during development. **Chrome DevTools** can help you inspect the DOM, debug JavaScript, monitor network requests, and troubleshoot layout issues.

As you work through the project, continuously review your code quality by asking yourself the following questions:

- Are your functions modular and reusable?
- Have you used descriptive variable and function names?
- Is your code logically grouped and easy to follow?
- Are comments used appropriately to explain important logic?
- Is your application responsive across different screen sizes?
- Does the interface clearly communicate with the user?
- Have you handled invalid input and failed API requests?

You should also ensure that your application follows [modern JavaScript best practices](#) and naming conventions.

Project overview

You will build a JavaScript web application that interacts with [TheMealDB API](#) to generate and manage “chef’s favorite meal” orders. The application will allow users to enter a main ingredient, retrieve matching meals from TheMealDB API, and generate an intelligent meal recommendation experience.

Your application must combine traditional front-end development with AI-enhanced interaction patterns. Although this project does not require integrating a large language model or generative AI API, it should still feel AI-enhanced through its interaction design, recommendation behavior, and dynamic user feedback.

Your application must:

- Retrieve meals dynamically from [TheMealDB API](#)
- Generate intelligent meal recommendations
- Simulate AI-style conversational feedback
- Provide responsive loading and processing states
- Handle invalid or failed requests gracefully
- Store and manage orders using `sessionStorage`
- Allow users to complete active orders
- Create a polished and responsive user experience

Understanding the AI-enhanced experience

Modern AI-powered applications often focus heavily on user interaction and responsiveness. Even when the underlying logic is relatively simple, the interface can feel significantly more intelligent when it:

- Provides dynamic feedback
- Explains system behavior clearly
- Handles uncertainty gracefully
- Gives contextual recommendations
- Simulates conversational interaction
- Responds naturally to user actions

In this project, your application should behave more like an intelligent assistant than a static ordering form.

For example, instead of simply displaying:

```
Meal selected: Beef Curry
```

your application could provide more conversational feedback such as:

```
The chef recommends Beef Curry today based on your selected ingredient.
```

Similarly, loading states should feel intentional and informative:

```
Searching the chef's recipe collection...
```

rather than generic or abrupt. This approach creates a more engaging and modern user experience.

TheMealDB API

Your application will use **TheMealDB API** to retrieve meals based on a main ingredient.

You will use the “**Filter by Ingredient**” endpoint:

```
https://www.themealdb.com/api/json/v1/1/filter.php?i=ingredient_name
```

For example:

```
https://www.themealdb.com/api/json/v1/1/filter.php?i=beef
```

The API returns a JSON object containing matching meals. If no meals are found, the API returns:

```
{  
  "meals": null  
}
```

Your application must detect and handle this situation properly.

Ingredient formatting requirements

TheMealDB API expects ingredient names in a specific format. Your application must convert **all ingredient** input to lowercase and replace spaces with underscores. For example:

User Input	API Format
Garlic Powder	garlic_powder
Chicken Breast	chicken_breast

This formatting process should happen automatically before the API request is sent.

Intelligent recommendation behavior

Once meal data has been retrieved successfully, your application must randomly select one meal as the “chef’s favorite recommendation”. This recommendation becomes the active order. Each order should contain:

Property	Description
Order number	Unique numeric identifier
Meal description	The selected meal name
Completion status	Complete or incomplete

The ingredient itself should not be stored as part of the order. To make the interaction feel more intelligent, your interface should present recommendations dynamically and conversationally. For example:

```
The chef analyzed your ingredient selection and recommends:  
Spicy Arrabiata Penne
```

This creates a more AI-inspired interaction flow.

AI-inspired interaction requirements

Your application must implement AI-enhanced user experience behaviors throughout the interface. These behaviors are now part of the project requirements.

Loading states

While the API request is processing, your application must:

- Display a visible loading state
- Disable relevant buttons
- Prevent duplicate requests
- Inform the user that the system is processing data

Examples include:

```
Analyzing ingredient selection...
```

```
Generating chef recommendation...
```

The loading state should disappear once the request completes or fails.

Intelligent feedback messages

Your interface should continuously communicate with the user throughout the interaction process. Feedback should feel contextual and user-friendly. For example:

Situation	Example Feedback
Successful recommendation	"The chef has selected a recommended meal for you."
Invalid ingredient	"No matching meals were found for that ingredient."
Failed request	"The recipe service is currently unavailable. Please try again."
Order completed	"Order #5 has been marked as completed."

Avoid abrupt or technical system messages.

Retry functionality

If the API request fails or returns invalid results, the user should be able to retry without refreshing the page. You may implement:

- A retry button
- Automatic retry logic
- Re-prompting for another ingredient
- A conversational recovery flow

The goal is to create a resilient and user-friendly experience.

Session storage requirements

Your application must use `sessionStorage` to persist order data during the browsing session. The following requirements must be met:

Order storage

All orders must be stored as a single JSON array. Example structure:

```
[
  {
    "orderNumber": 1,
    "description": "Beef Curry",
    "completed": false
  }
]
```

Last order number

The latest order number must be stored separately in `sessionStorage`. This prevents the need to loop through all orders when generating new order numbers. Example:

```
sessionStorage.setItem(
  "lastOrderNumber",
  5
);
```

Displaying active orders

Your application must display all incomplete orders currently stored in `sessionStorage`. Each displayed order should include:

- Order number
- Meal description
- Completion status indicator

You may choose the interface layout and styling, but the information must be clear and readable.

Completing orders

Users must be able to mark an order as completed. Your application should:

1. Allow the user to enter an order number
2. Verify whether the order exists
3. Update the completion status
4. Save the updated data back into `sessionStorage`
5. Refresh the displayed incomplete orders

If the order number does not exist, the application must display an appropriate feedback message.

You have creative freedom regarding the visual design of the application, but the final result should appear polished, modern, and responsive. The goal is to create a functional and accessible application, but how you choose to design it is up to you.



Capstone project

In this capstone project, you will build an AI-enhanced meal ordering web application using HTML, CSS, JavaScript, **TheMealDB API**, and `sessionStorage`.

Your application must demonstrate asynchronous API handling, DOM manipulation, intelligent recommendation behavior, responsive user experience design, and persistent client-side storage management.

1. Create a complete web application using:
 - HTML
 - CSS
 - JavaScript
 - TheMealDB API
 - `sessionStorage`
2. Build a meal recommendation workflow.
 - Allow users to enter a main ingredient.
 - Format the ingredient correctly for the API request.
 - Retrieve meals using TheMealDB.
 - Randomly select a “chef’s favorite” meal recommendation.
 - Present the recommendation using conversational or AI-inspired feedback.
3. Implement AI-enhanced UX behaviors.
 - Add loading states during API requests.
 - Disable buttons during processing.
 - Display meaningful feedback messages.
 - Handle failed requests gracefully.
 - Add retry functionality for failed or invalid requests.
4. Store and manage orders using `sessionStorage`.
 - Store all orders in a JSON array.
 - Store the latest order number separately.
 - Ensure all order numbers are unique.
 - Persist completion status updates correctly.

5. Display and manage active orders.
 - Display only incomplete orders.
 - Allow users to mark orders as completed.
 - Update `sessionStorage` after changes.
 - Handle invalid order numbers appropriately.
6. Create a responsive and professional user interface.
 - Use responsive CSS techniques.
 - Ensure readability and accessibility.
 - Organize content clearly.
 - Design a visually consistent experience.
7. Refactor and document your code.
 - Use meaningful naming conventions.
 - Organize logic into reusable functions.
 - Add comments where appropriate.
 - Remove unused code and redundant logic..

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
